

Perceptron

- I) Por que uma rede de Perceptrons de Rosenblatt não pode ser treinada via descida de gradiente e *backpropagation*?
- II) Prove que um Perceptron com duas entradas $\hat{y} = \text{sin}(\theta_0 + X_1\theta_1 + X_2\theta_2)$ gera uma fronteira de decisão linear.
- III) Crie um Perceptron que se comporta como uma função *and*.
- IV) Demonstre que uma regularização L2 na função de custo de uma unidade com função de ativação identidade é equivalente a diminuir a magnitude do vetor de pesos antes de realizar a atualização. *Dica: escreva a função de custo com o termo de regularização $\|\theta\|^2$ e use essa função de custo na expressão de atualização dos pesos da descida de gradiente. Colocando o θ em evidência ficará claro que os pesos estão sendo multiplicados por um valor na faixa (0, 1).*

Descida de Gradiente e Backpropagation

- V) Dado que a descida de gradiente possui na sua formalização original a seguinte regra de atualização $\theta_{t+1} = \theta_t + \eta \nabla_{\theta} J$. Por que a atualização dos pesos com $-\nabla_{\theta} J$ diminui o valor da função de custo?
- VI) Por que dizemos que a formulação original do *momentum* calcula o gradiente “no local errado”?
- VII) Qual o efeito prático de normalizar o gradiente (conforme feito no Adam)?
- VIII) Considere um MLP com uma unidade na entrada, uma unidade na saída, K camadas ocultas e D unidades em cada camada. Quantos parâmetros esse MLP possui?
- IX) Quais são os quatro passos realizados para fazer o treinamento de um MLP?
- X) Qual a diferença de uma iteração para uma época?
- XI) Considerando que θ_t denota o vetor de parâmetros treináveis de uma rede neural na iteração t e as seguintes informações:

$$\eta = 0.1, \quad \beta = 0.8, \quad \theta_0 = \begin{bmatrix} 5 \\ 10 \end{bmatrix}, \quad \nu_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$
$$\nabla_{\theta_0} J = \begin{bmatrix} -2 \\ 5 \end{bmatrix}, \quad \nabla_{\theta_1} J = \begin{bmatrix} -1 \\ 8 \end{bmatrix}, \quad \nabla_{\theta_2} J = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

- a) Como fica θ_1 ao seguir a regra de atualização do SGD?
- b) Como fica θ_1 ao seguir a regra de atualização da descida de gradiente com *normalização*?
- c) Como fica θ_2 ao seguir a regra de atualização da descida de gradiente com *momentum*?
- d) 🦴 Como fica θ_2 ao seguir a regra de atualização do otimizador ADAM? (Nota: ignore o $\beta = 0.8$ usado nos itens anteriores; aqui usamos os hiperparâmetros padrão do Adam, $\beta_1 = 0.9$ e $\beta_2 = 0.999$)

Funções de Ativação

XII) Calcule as derivadas das seguintes funções de ativação:

a) ReLU $\varphi(x) = \max(0, x)$

b) Sigmoid $\varphi(x) = \frac{1}{(1+e^{-x})}$

c)  Softmax $\varphi(\mathbf{x}) = S_i = \frac{e^{x_i}}{\sum e^{x_j}}$, (encontrar a matriz Jacobiana)

XIII) Qual a utilidade das funções de ativação nas camadas ocultas de um MLP?

XIV) Qual a utilidade das funções de ativação na camada de saída de um MLP?

Inicialização de Pesos

XV) Que problemas podemos ter com unidades na camada oculta que usam a função sigmoid?

XVI) Que problemas podemos ter com unidades na camada oculta que usam a função relu?

XVII) Qual a motivação das inicializações *He* e *Xavier*.

Funções de Custo

XVIII) Calcule as derivadas das seguintes funções de custo:

a) $L_{l2} = \sum (y^{(i)} - \hat{y}^{(i)})^2$

b) $L_{BCE} = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$

XIX) Desenvolva a função de custo do Erro Médio Quadrático a partir da distribuição Gaussiana.

$$Pr(y|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \mu)^2}{2\sigma^2}\right)$$

XX) Desenvolva a função de custo Entropia Binária Cruzada a partir da distribuição de Bernoulli.

$$Pr(y|\lambda) = (1 - \lambda)^{(1-y)} \lambda^y$$

XXI)  Suponha que você quer criar uma rede neural para estimar a direção y (em radianos) do vento a partir de medições de pressão $\mathbf{x}$. Uma distribuição que trabalha no domínio circular é a distribuição de von Mises:

$$Pr(y|\mu, k) = \frac{\exp[k * \cos(y - \mu)]}{2\pi * Bessel_0[k]}$$

onde μ é a direção média e k é o inverso da variância. O termo $Bessel_0[k]$ é uma função modificada de Bessel com grau 0. Crie uma função de custo para aprender o parâmetro μ dessa distribuição.

- XXII) 🦴 Suponha que você quer criar uma rede neural para estimar o número de pedestres $y \in \{0, 1, 2, \dots\}$ que passam em uma rua da cidade em algum determinado momento. Você possui um vetor de atributos x que contém informações sobre a hora do dia, sobre o clima e sobre o dia ser feriado ou não. Uma distribuição de probabilidade adequada para trabalhar com esse problema é a distribuição de Poisson:

$$Pr(y = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

onde λ é o único parâmetro, que representa a média da distribuição. Crie uma função de custo assumindo que temos acesso a I instâncias de treinamento compostas por pares $\{x^i, y^i\}$.

Backpropagation: Cálculo Manual

- XXIII) Considerando a rede da Figura 1, resolva os itens abaixo:

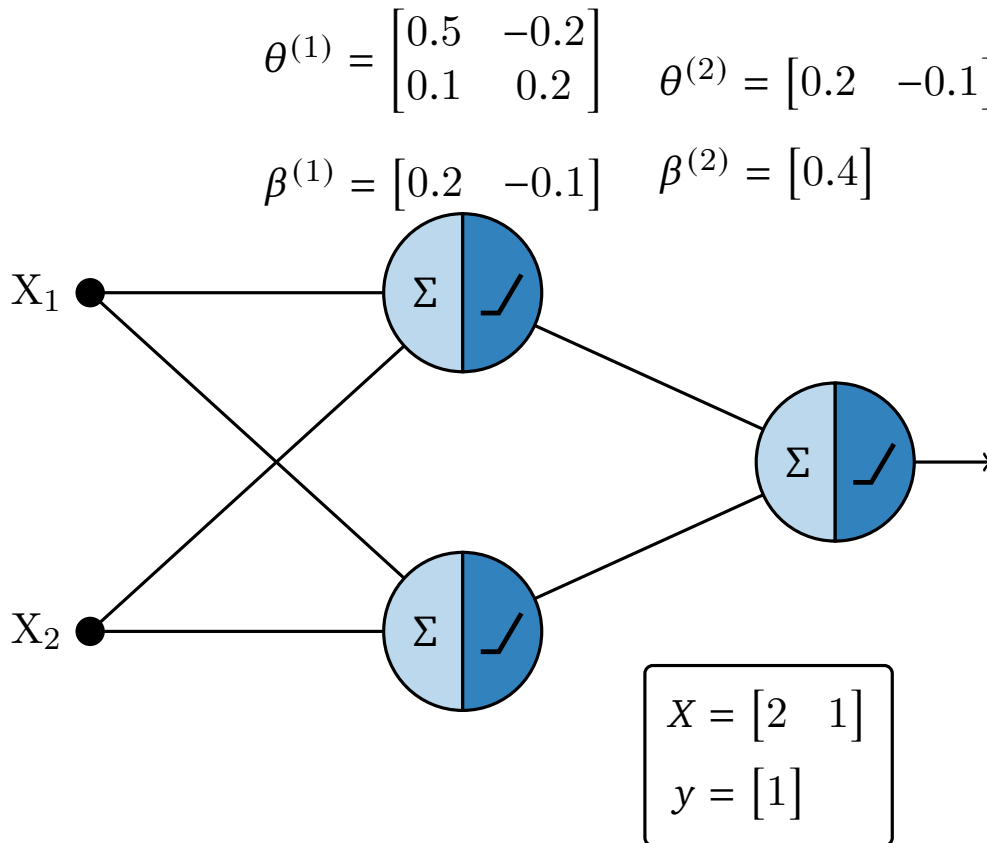


Figura 1: Multilayer Perceptron

- Compute o *Forward Pass*.
- Compute o valor da Loss $J = \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2$.
- Compute o *Backward Pass*.
- Realize a atualização de todos os parâmetros treináveis da rede conforme o otimizador SGD $\theta_{t+1} = \theta_t - \eta \nabla J$.

Gabarito

- I) Pois a derivada da função de ativação sinal é zero ou indefinida em todo domínio da função. Devido a isso, não conseguimos propagar os erros.
- II) Para resolver essa questão você deve mostrar que $\theta_0 + X_1\theta_1 + X_2\theta_2 = 0$ é uma reta.
- III) $\hat{y} = \text{sinhal}(\theta_0 + X_1\theta_1 + X_2\theta_2)$, onde $\theta_0 = -1$, $\theta_1 = 1$, $\theta_2 = 1$ e as entradas falsas são codificadas como -1 e as entradas verdadeiras como $+1$.
- IV) $\theta_{(t+1)} = (1 - 2\eta\lambda)\theta_t - \eta\nabla_{\theta}J$, onde λ é a constante de regularização L2.
- V) O gradiente nos dá a direção de atualização dos pesos que causaria aumento na função de custo. Ao atualizar os pesos em direção contrária, estamos diminuindo o custo.
- VI) Dizemos que o gradiente está atrasado pois, devido ao *momentum*, já existe uma atualização dos pesos conforme o histórico dos gradientes. Essa observação leva a definição do *Nesterov Accelerated Momentum*.
- VII) Ao normalizar o gradiente, fica mais fácil definir uma taxa de aprendizado que funcione igualmente bem para todos os parâmetros da rede.
- VIII) $3D+1+(K-1)D(D+1)$. Repare que essa expressão não é a única resposta possível para o exercício. Dependendo de como você desenvolver a questão poderá chegar em expressões equivalentes como por exemplo: $2D + (K-1)DD + KD + 1$
- IX) *Forward Pass*: onde as saídas são calculadas; *Loss Function*: onde a diferença entre a saída e o valor esperado são comparadas; *Backward Pass*: onde os gradientes da função de custo em relação a cada um dos pesos é calculado; *Optimizer*: onde os pesos são atualizados.
- X) Uma iteração consiste na apresentação de algumas instâncias para o modelo, sob as quais são realizados *Forward Pass*, cálculo da função de custo, *Backward Pass* e otimização. Uma época ocorre quando foram feitas iterações suficientes para ver todas as instâncias do conjunto de treinamento 1 vez.

XI)

$$\eta = 0.1, \quad \theta_0 = \begin{bmatrix} 5 \\ 10 \end{bmatrix}, \quad v_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

$$\nabla_{\theta_0}J = \begin{bmatrix} -2 \\ 5 \end{bmatrix}, \quad \nabla_{\theta_1}J = \begin{bmatrix} -1 \\ 8 \end{bmatrix}, \quad \nabla_{\theta_2}J = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

a)

$$\theta_1 = \begin{bmatrix} 5 \\ 10 \end{bmatrix} - 0.1 \begin{bmatrix} -2 \\ 5 \end{bmatrix} = \begin{bmatrix} 5.2 \\ 9.5 \end{bmatrix}$$

b)

$$\theta_1 = \begin{bmatrix} 5 \\ 10 \end{bmatrix} - 0.1 \left(\begin{bmatrix} -2 \\ 5 \end{bmatrix} \div \begin{bmatrix} 2 \\ 5 \end{bmatrix} \right) = \begin{bmatrix} 5.1 \\ 9.9 \end{bmatrix}$$

c)

$$\begin{aligned}
 v_1 &= 0.8 \begin{bmatrix} 0 \\ 0 \end{bmatrix} + (1 - 0.8) \begin{bmatrix} -2 \\ 5 \end{bmatrix} = \begin{bmatrix} -0.4 \\ 1 \end{bmatrix} \\
 \theta_1 &= \begin{bmatrix} 5 \\ 10 \end{bmatrix} - 0.1 \begin{bmatrix} -0.4 \\ 1 \end{bmatrix} = \begin{bmatrix} 5.04 \\ 9.9 \end{bmatrix} \\
 v_2 &= 0.8 \begin{bmatrix} -0.4 \\ 1 \end{bmatrix} + (1 - 0.8) \begin{bmatrix} -1 \\ 8 \end{bmatrix} = \begin{bmatrix} -0.52 \\ 2.40 \end{bmatrix} \\
 \theta_2 &= \begin{bmatrix} 5.04 \\ 9.9 \end{bmatrix} - 0.1 \begin{bmatrix} -0.52 \\ 2.40 \end{bmatrix} = \begin{bmatrix} 5.092 \\ 9.660 \end{bmatrix}
 \end{aligned}$$

d) 🦴 Considerando as regras de atualização do ADAM com $\eta = 0.1$, $\beta = 0.9$ e $\gamma = 0.999$, podemos calcular o valor de θ_2 .

Primeiro, calculamos θ_1 para a iteração $t = 1$:

Momento (m_1):

$$m_1 = \beta m_0 + (1 - \beta) \nabla_{\theta_0} J = 0.9 \begin{bmatrix} 0 \\ 0 \end{bmatrix} + (1 - 0.9) \begin{bmatrix} -2 \\ 5 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + 0.1 \begin{bmatrix} -2 \\ 5 \end{bmatrix} = \begin{bmatrix} -0.2 \\ 0.5 \end{bmatrix}$$

Termo normalizador (v_1):

$$v_1 = \gamma v_0 + (1 - \gamma) (\nabla_{\theta_0} J)^2 = 0.999 \begin{bmatrix} 0 \\ 0 \end{bmatrix} + (1 - 0.999) \begin{bmatrix} (-2)^2 \\ 5^2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + 0.001 \begin{bmatrix} 4 \\ 25 \end{bmatrix} = \begin{bmatrix} 0.004 \\ 0.025 \end{bmatrix}$$

Correção de viés (*bias correction*):

$$\begin{aligned}
 \hat{m}_1 &= \frac{m_1}{1 - \beta^1} = \frac{\begin{bmatrix} -0.2 \\ 0.5 \end{bmatrix}}{1 - 0.9} = \begin{bmatrix} -2 \\ 5 \end{bmatrix} \\
 \hat{v}_1 &= \frac{v_1}{1 - \gamma^1} = \frac{\begin{bmatrix} 0.004 \\ 0.025 \end{bmatrix}}{1 - 0.999} = \begin{bmatrix} 4 \\ 25 \end{bmatrix}
 \end{aligned}$$

Atualização de θ_1 :

$$\theta_1 = \theta_0 - \eta \frac{\hat{m}_1}{\sqrt{\hat{v}_1} + \epsilon} = \begin{bmatrix} 5 \\ 10 \end{bmatrix} - 0.1 \begin{bmatrix} \frac{-2}{\sqrt{4}} \\ \frac{5}{\sqrt{25}} \end{bmatrix} = \begin{bmatrix} 5 \\ 10 \end{bmatrix} - 0.1 \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 5.1 \\ 9.9 \end{bmatrix}$$

Agora, calculamos θ_2 para a iteração $t = 2$:

Momento (m_2):

$$m_2 = \beta m_1 + (1 - \beta) \nabla_{\theta_1} J = 0.9 \begin{bmatrix} -0.2 \\ 0.5 \end{bmatrix} + (1 - 0.9) \begin{bmatrix} -1 \\ 8 \end{bmatrix} = \begin{bmatrix} -0.18 \\ 0.45 \end{bmatrix} + 0.1 \begin{bmatrix} -1 \\ 8 \end{bmatrix} = \begin{bmatrix} -0.28 \\ 1.25 \end{bmatrix}$$

Termo normalizador (v_2):

$$v_2 = \gamma v_1 + (1 - \gamma) (\nabla_{\theta_1} J)^2 = 0.999 \begin{bmatrix} 0.004 \\ 0.025 \end{bmatrix} + (1 - 0.999) \begin{bmatrix} (-1)^2 \\ 8^2 \end{bmatrix} = \begin{bmatrix} 0.003996 \\ 0.024975 \end{bmatrix} + 0.001 \begin{bmatrix} 1 \\ 64 \end{bmatrix} = \begin{bmatrix} 0.004996 \\ 0.088975 \end{bmatrix}$$

Correção de viés (*bias correction*):

$$\hat{m}_2 = \frac{m_2}{1 - \beta^2} = \frac{\begin{bmatrix} -0.28 \\ 1.25 \end{bmatrix}}{1 - 0.9^2} = \frac{\begin{bmatrix} -0.28 \\ 1.25 \end{bmatrix}}{0.19} \approx \begin{bmatrix} -1.47368 \\ 6.57895 \end{bmatrix}$$

$$\hat{v}_2 = \frac{v_2}{1 - \gamma^2} = \frac{\begin{bmatrix} 0.004996 \\ 0.088975 \end{bmatrix}}{1 - 0.999^2} = \frac{\begin{bmatrix} 0.004996 \\ 0.088975 \end{bmatrix}}{0.001999} \approx \begin{bmatrix} 2.49925 \\ 44.50975 \end{bmatrix}$$

Atualização de θ_2 :

$$\theta_2 = \theta_1 - \eta \frac{\hat{m}_2}{\sqrt{\hat{v}_2} + \epsilon} = \begin{bmatrix} 5.1 \\ 9.9 \end{bmatrix} - 0.1 \frac{\begin{bmatrix} -1.47368 \\ 6.57895 \end{bmatrix}}{\begin{bmatrix} \sqrt{2.49925} \\ \sqrt{44.50975} \end{bmatrix}} \approx \begin{bmatrix} 5.1 \\ 9.9 \end{bmatrix} - 0.1 \begin{bmatrix} -0.9322 \\ 0.9869 \end{bmatrix} = \begin{bmatrix} 5.1932 \\ 9.8013 \end{bmatrix}$$

Portanto, o valor de θ_2 ao seguir a regra de atualização do ADAM é:

$$\theta_2 \approx \begin{bmatrix} 5.193 \\ 9.801 \end{bmatrix}$$

XII) a)

$$\varphi'(x) = \begin{cases} 0, & x \leq 0, \\ 1, & x > 0 \end{cases}$$

b) $\varphi'(x) = \varphi(x)(1 - \varphi(x))$

c) 🦴

$$\frac{\partial S_i}{\partial x_j} = \begin{cases} S_i(1 - S_i), & i = j, \\ -S_i S_j, & i \neq j. \end{cases}$$

Forma compacta: $\frac{\partial S_i}{\partial x_j} = S_i (\delta_{ij} - S_j)$, onde δ_{ij} é o delta de Kronecker.

XIII) Adicionar não linearidades.

XIV) Ajustar a saída da rede neural a imagem da função modelada

XV) *Vanishing Gradient*

XVI) Unidades mortas

XVII) Inicializar os pesos da rede neural de modo que não ocorra explosão nem dissipação de gradiente.

XVIII) a) $2(\hat{y} - y)$

b) $\frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$ (esta é $\partial L_{BCE} / \partial \hat{y}$, não $\partial L_{BCE} / \partial \theta$; ao multiplicar pela derivada da sigmoid $\hat{y}(1 - \hat{y})$ e por X^T , simplifica para $X^T(\hat{y} - y)$)

XIX) $L_{l2} = \sum (y^{(i)} - \hat{y}^{(i)})^2$

XX) $L_{BCE} = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$

XXI) 🦴 $-\sum_i \cos(y^{(i)} - f(x^{(i)}, \theta))$ (a NLL é $-\sum_i \ln \Pr(y^{(i)} | \mu, k)$. O denominador $2\pi \cdot \text{Bessel}_0[k]$ e o k não dependem de $\mu = f(x^{(i)}, \theta)$, então caem como constantes; sobra apenas o termo $-k \sum_i \cos(y^{(i)} - f(x^{(i)}, \theta))$, que com k fixo é proporcional ao apresentado)

XXII)  $\sum_{i=1}^I (f[x^{(i)}, \theta] - y^{(i)} \ln(f[x^{(i)}, \theta]))$

XXIII) **Backpropagation: Cálculo Manual.** Considerando ReLU $\varphi(z) = \max(0, z)$ em todas as unidades (inclusive na saída) e taxa de aprendizado $\eta = 0.1$.

Forward Pass:

$$\begin{aligned} z^{(1)} &= \theta^{(1)} X + \beta^{(1)} = \begin{bmatrix} 0.5 & -0.2 \\ 0.1 & 0.2 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix} = \begin{bmatrix} 1.0 \\ 0.3 \end{bmatrix}, \quad a^{(1)} = \varphi(z^{(1)}) = \begin{bmatrix} 1.0 \\ 0.3 \end{bmatrix} \\ z^{(2)} &= \theta^{(2)} a^{(1)} + \beta^{(2)} = \begin{bmatrix} 0.2 & -0.1 \end{bmatrix} \begin{bmatrix} 1.0 \\ 0.3 \end{bmatrix} + 0.4 = 0.57, \quad \hat{y} = a^{(2)} = \varphi(0.57) = 0.57 \end{aligned}$$

Loss: $J = (\hat{y} - y)^2 = (0.57 - 1)^2 = 0.1849$.

Backward Pass: como $\frac{\partial J}{\partial \hat{y}} = 2(\hat{y} - y) = -0.86$ e $\varphi'(z) = 1$ para $z > 0$,

$$\begin{aligned} \delta^{(2)} &= 2(\hat{y} - y) \odot \varphi'(z^{(2)}) = -0.86 \\ \frac{\partial J}{\partial \theta^{(2)}} &= \delta^{(2)} a^{(1)\top} = \begin{bmatrix} -0.86 & -0.258 \end{bmatrix}, \quad \frac{\partial J}{\partial \beta^{(2)}} = \delta^{(2)} = -0.86 \\ \delta^{(1)} &= (\delta^{(2)} \theta^{(2)}) \odot \varphi'(z^{(1)}) = \begin{bmatrix} -0.172 & 0.086 \end{bmatrix} \\ \frac{\partial J}{\partial \theta^{(1)}} &= \delta^{(1)\top} X^\top = \begin{bmatrix} -0.172 \\ 0.086 \end{bmatrix} \begin{bmatrix} 2 & 1 \end{bmatrix} = \begin{bmatrix} -0.344 & -0.172 \\ 0.172 & 0.086 \end{bmatrix}, \quad \frac{\partial J}{\partial \beta^{(1)}} = \delta^{(1)} = \begin{bmatrix} -0.172 & 0.086 \end{bmatrix} \end{aligned}$$

Atualização (SGD, $\theta_{t+1} = \theta_t - \eta \nabla J$):

$$\begin{aligned} \theta^{(1)} &\leftarrow \begin{bmatrix} 0.5 & -0.2 \\ 0.1 & 0.2 \end{bmatrix} - 0.1 \begin{bmatrix} -0.344 & -0.172 \\ 0.172 & 0.086 \end{bmatrix} = \begin{bmatrix} 0.5344 & -0.1828 \\ 0.0828 & 0.1914 \end{bmatrix} \\ \beta^{(1)} &\leftarrow \begin{bmatrix} 0.2 & -0.1 \end{bmatrix} - 0.1 \begin{bmatrix} -0.172 & 0.086 \end{bmatrix} = \begin{bmatrix} 0.2172 & -0.1086 \end{bmatrix} \\ \theta^{(2)} &\leftarrow \begin{bmatrix} 0.2 & -0.1 \end{bmatrix} - 0.1 \begin{bmatrix} -0.86 & -0.258 \end{bmatrix} = \begin{bmatrix} 0.286 & -0.0742 \end{bmatrix} \\ \beta^{(2)} &\leftarrow 0.4 - 0.1(-0.86) = 0.486 \end{aligned}$$